

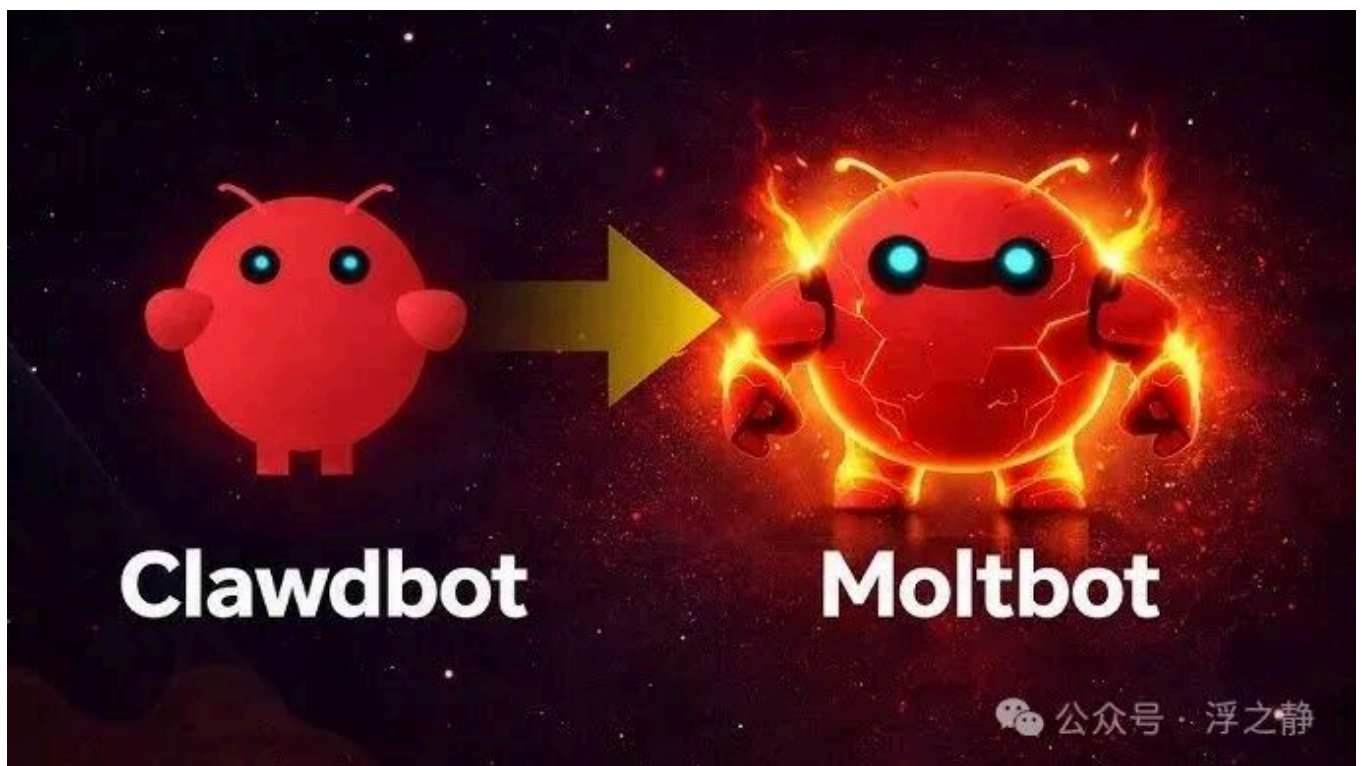
初识 Moltbot (原名 Clawdbot)

mp.weixin.qq.com/s/x9LaUHjx7JFmsaptDvWPQQ

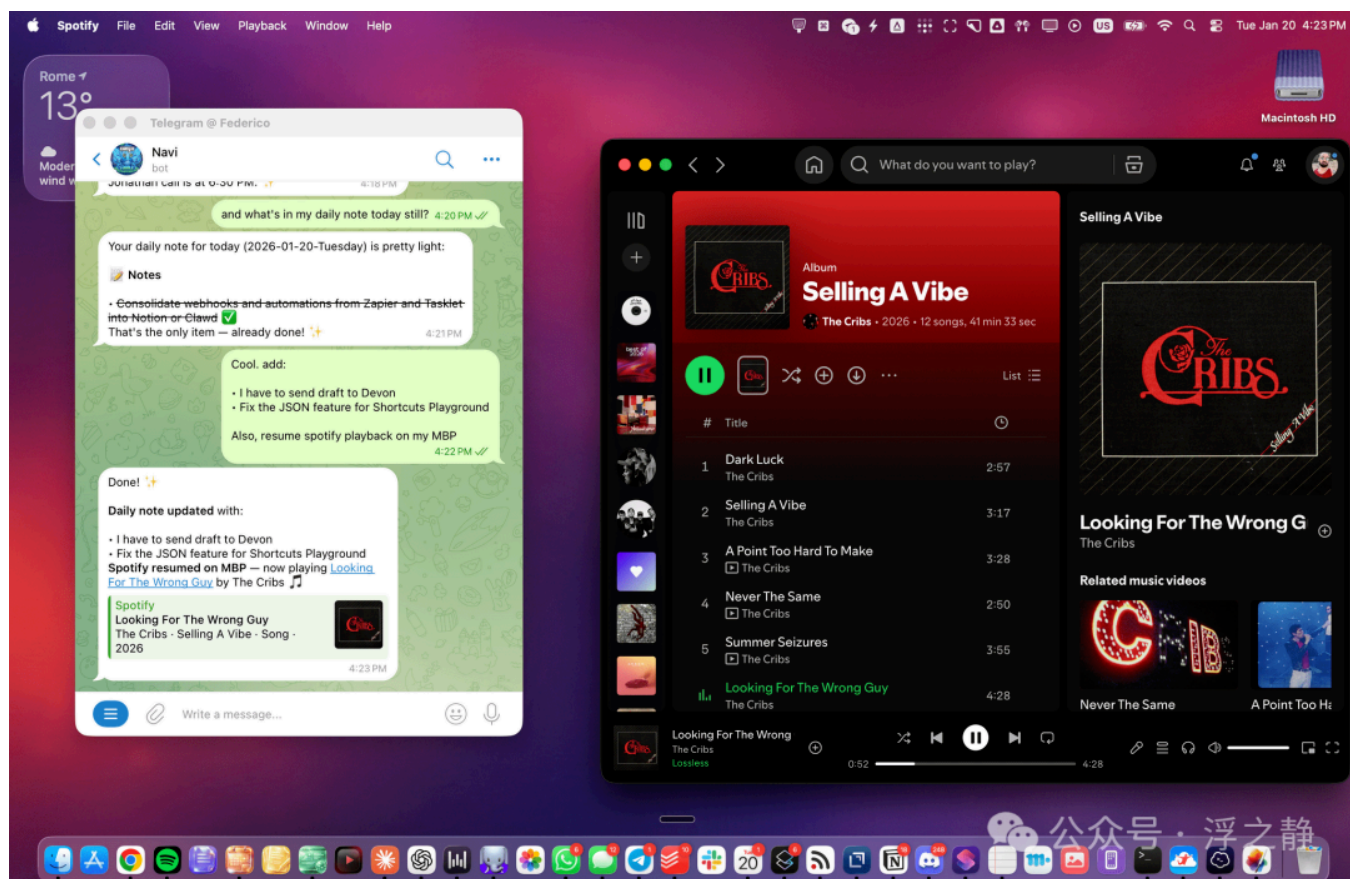
lencx

Moltbot 是多个前沿 AI 架构 (如 MCP、Skills、ACP^[1]、A2UI^[2] 等) 的混合体, 不适合靠读几篇文章速成, 唯有反复阅读文档、issues、动手实践才能真正理解其理念。它未必是最完美的智能体设计, 但无疑是当前最具探索精神的开源混合代理框架。

不了解 Skills、MCP 的可以看这两篇文章: [从 Prompt Engineering 到 Context Engineering](#)、[深度解析: Anthropic MCP 协议](#), 也可以读读这篇[深度解析: Claude Code Cowork](#), Moltbot 是比 Cowork 更先进的设计。



Moltbot 本质上是一个常驻运行的 **Gateway**: 把你在 WhatsApp / Telegram / Discord / iMessage 等消息应用里的消息路由给后端 agent, 再把结果回写到同一线程里, 让 AI 直接“住进”你日常用的 app, 而不是你在网页聊天框和各个工具之间来回复制粘贴。它的价值不在“更会聊”, 而在“更像助手”: 对话会沉淀为会话/工作区状态, 并可通过心跳/定时任务做谨慎的“主动推送”(可关闭)。更关键的是, 它试图把 AI 能力下沉到你的本地环境, 通过统一网关把消息渠道、文件系统、终端、浏览器等执行面打通, 绕开传统 SaaS 各自为政的接口与数据孤岛, 把零散自动化收拢成可组合的工作流。大概长这样, 截图来自网络。



它到底能“干多少活”，取决于你给它接入了哪些本机工具权限（命令/文件/脚本等），权限越大风险越高；官方也因此强调默认 loopback 监听、不要把控制面暴露公网。硬件上 Mac mini 不是刚需，Node 服务跑在本机、家用小主机或便宜 VPS 都行，主要成本在模型调用与任务强度。与此同时，这种把系统级权限交给模型的设计天然更敏感：一旦控制台误暴露且缺少认证，就可能引发未授权访问乃至更严重的执行风险（如 PR #1795^[3]，现已合并）。

🧠 安全警告

不建议技术小白把 Moltbot 直接装在日常工作机上。它通常需要访问文件系统、终端、网络等高权限资源；一旦被误配置、被诱导执行工具，代价可能不是“泄露几份文件”，而是主机级别的损害。更稳妥的做法是在新主机或隔离环境（虚拟机/容器/专用账号）里先试验，跑通后再逐步收紧权限与暴露面。

同时要有一个现实认知：**整个 IPv4 互联网几乎是在被持续扫描的**——不论是白帽还是黑产都会扫。像 Shodan^[4]、Censys^[5] 这类服务会把“能响应的公网主机”收录进可搜索数据库，并按端口/banner/证书/HTTP 响应内容等特征建立索引。换句话说，你的服务只要对公网可达，很快就会“被看到”。

[Explore](#)
[Pricing](#)

[Login](#)

TOTAL RESULTS

1,008

TOP COUNTRIES

United States	266
Germany	226
United Arab Emirates	146
Finland	136
Singapore	39
More...	

TOP PORTS

5353	992
443	5
51235	2
80	1
3389	1
More...	

[View Report](#)
[Browse Images](#)
[View on Map](#)
[Advanced Search](#)

Access Granted: Want to get more out of your existing Shodan account? Check out [everything you have access to.](#)

159.69.27.118

static:118.27.89.159.clients.your-ser
ver.de
Hetzner Online GmbH
Germany, Nürnberg

39.96.175.54

Aliyun Computing Co., LTD
China, Beijing

89.167.3.165

static:165.3.167.89.clients.your-ser
er.de
Hetzner Online GmbH
Germany, Gunzenhausen

mDNS:

services:

18789/tcp clawdbot-gw:
role=gateway
gatewayPort=18789
lanHost=ubuntu-4gb-nbg1-1.local
displayName=ubuntu-4gb-nbg1-1
tailnetDns=ubuntu-4gb-nbg1-1.tail20245f.ts.net
cliPath=/home/clawdbot/clawdbot/dist/entry.js
sshPort=22
transpo...

2026-01-28T06:33:10.830597

2026-01-28T06:32:39.491999

2026-01-28T06:26:42.687403

[Log in](#)
[Register](#)

[Search Results](#)
[Report Builder](#)

ASSET TYPES

Hosts	1.1K
Certificates	446
Web Properties	2.6K

SOFTWARE VENDORS

f5	1.7K
cloudflare	1.6K
openbsd	757
traefik_labs	292
openresty	252
More	

SOFTWARE PRODUCTS

nginx	1.7K
cloudflare_load_balancer	816
waf	816
openssh	757
traefik_proxy	292

8.219.205.145: 18789 = WEB PROPERTY

AS OF 2026年1月28日 | UTC 06:20

HTML Title

Clawdbot Control

Browser Trust

No Data

Ever Valid?

Unknown

Endpoints (1)

18789 / HTTP

MATCHED FIELDS

web.endpoints.http.body <title>Clawdbot Control</>
web.endpoints.http.html_tags <title>Clawdbot Control</>
web.endpoints.http.html_title Clawdbot Control

110.238.78.229: 18789 = WEB PROPERTY

AS OF 2026年1月28日 | UTC 06:08

HTML Title

Clawdbot Control

Browser Trust

No Data

Ever Valid?

Unknown

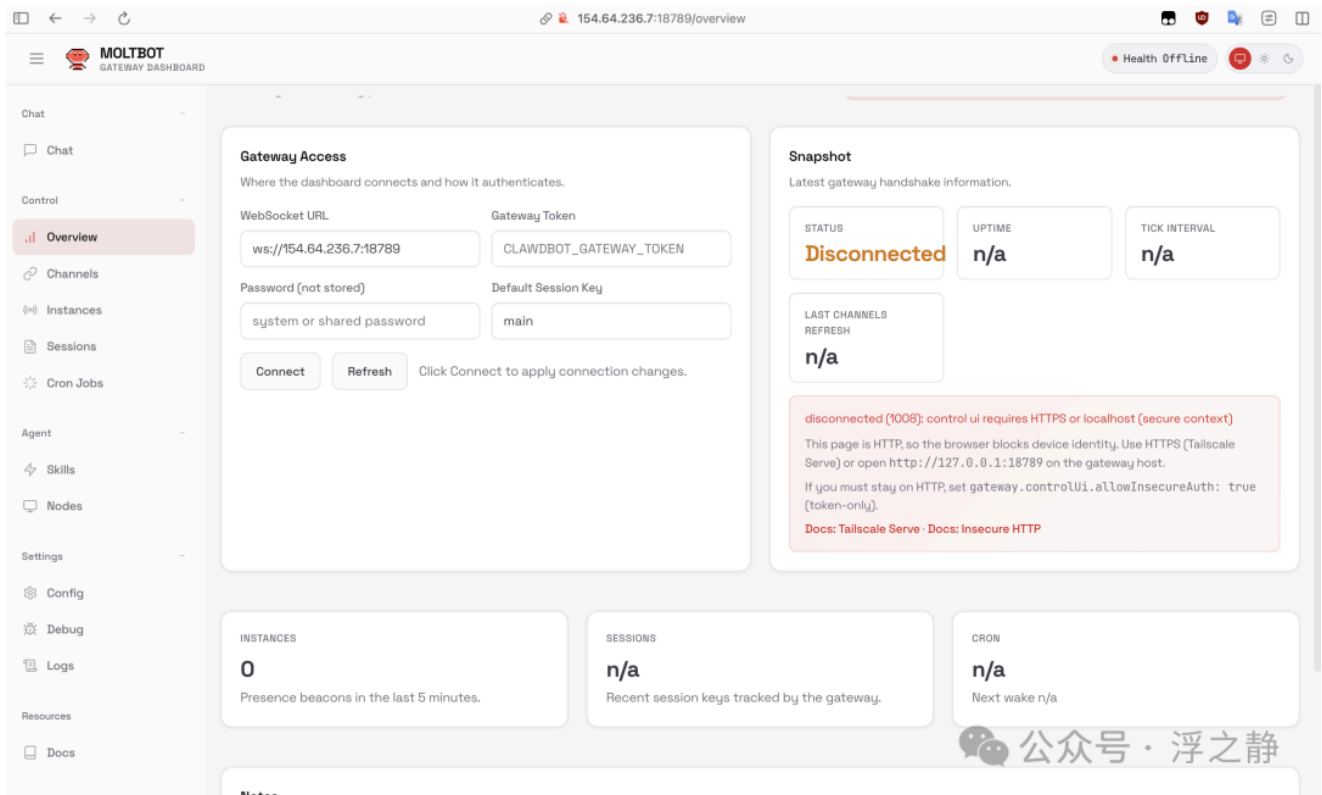
Endpoints (1)

18789 / HTTP

MATCHED FIELDS

web.endpoints.http.body <title>Clawdbot Control</>
web.endpoints.http.html_tags <title>Clawdbot Control</>
web.endpoints.http.html_title Clawdbot Control

对 Moltbot/Clawdbot 来说，暴露 Gateway 当然不妙，但暴露 Control UI 更危险：Control UI 往往意味着更直接的管理入口、更明确的交互界面，也更容易被针对。更糟的是，是否会被搜到并不取决于你“是否出名”，而取决于你的服务是否存在可检索的指纹：比如 HTML 里某些稳定的关键词组合、<title> 文本、独特的 favicon/icon、固定路径的静态资源名、端口号 18789 等。只要 HTTP 响应里有任何独特特征，别人就能在 Shodan/Censys 上写出查询，通常上线几小时内就能拉出一批公网实例列表。以下截图就是暴露在公网的 Moltbot 实例：



因此，如果你把网关/控制台直接绑到公网，又没有认证、白名单、反向代理防护，基本等价于“在公网上裸奔”：不是“会不会被找到”，而是“什么时候被找到、谁先动手”。

背景

过去的生成式 AI 更像“会聊天的搜索框”，你得去网页里问、再把结果搬去别的地方用。现在讨论的重点逐渐变成 **Agentic AI**：模型不只负责回答，还要能在你的权限边界内调用工具、完成任务、持续运行。这会把软件形态从“单次对话”推向“常驻服务 + 可执行链路”，也迫使架构把执行、权限、审计、隔离这些老问题重新摆到台面上。

在这条线上，Moltbot^[6]（前身 Clawdbot）是近期爆火的一个开源样本：**本地优先（Local-First）+ 可自托管**的代理框架。它由奥地利开发者 Peter Steinberger^[7]于 2025 年底发起，热度在 2026 年初快速攀升（GitHub Stars 已超 8 万），迅速变成开发者社区的“试验田”。



Clawdbot 最初的定位很直白：“**Claude with hands (长了手的 Claude)**”。它想表达的不是“又一个聊天 UI”，而是：把 LLM 接到可执行工具上，让它能在你常用的消息应用（WhatsApp / Telegram / Slack / iMessage...）里接收指令、调用本机/外部系统能力（文件、终端、浏览器、脚本等），再把结果回写到同一条对话线程里。换句话说，AI 不用你搬运，它直接“住进”你的工作流入口。

随着项目传播，品牌合规问题也随之而来：由于名称与 Anthropic Claude 发音/联想过近，项目方被要求更名。团队没有硬扛，而是顺势完成了一次“蜕壳式”重塑：**Moltbot**（molt = 甲壳类蜕壳）。这次更名短期带来了一些现实混乱（链接迁移、社媒账号被抢注/诈骗趁乱出现等），但也让项目从“某个模型的外壳”更明确地转向独立的代理框架品牌：你可以换模型、换渠道、换工具，但核心是你自己掌控运行环境与行为边界。



龙虾起源

Moltbot 的“龙虾宇宙”本质是社区包装出来的一套叙事壳，方便记忆也方便传播：项目最早从一个 WhatsApp 网关（Warelay）起步，逐渐演化出带人格的“Clawd”，更名后变成“Molty”，社区把这次改名叫 **The Great Molt**，口号是 **“New shell, same soul”**。如果你想看完整设定，官方文档有专门的 Moltbot Lore^[8]。

Moltbot 崛起

硬件回归：从云到壳，AI 的栖居之地

Moltbot 的走红带来一个很现实的副作用：很多人开始认真折腾“常驻主机”，Mac mini（尤其是 M 系列）因此成了社区里的高频选项，甚至被戏称为“Mac mini 复兴”。它反映的不是某个硬件突然更强了，而是 **AI 的形态变了**：推理可以在云上，但“长期在线 + 可执行”这部分必须落在一台你能控制、能一直开着的机器上。



Alex Finn @AlexFinn · Jan 25

Yesterday I installed **CloudBot** on this mac mini. An AI agent assistant that



jaack @ijaack94 · 20h

/@moltbot is even quite light on resources!



本地优先对硬件的真实要求

Moltbot 属于典型的 Local-First：模型调用在云端，**代理运行时**（记忆、技能调度、浏览器/文件/脚本等工具执行）却在本地。于是硬件需求变得很朴素也很苛刻：

始终在线 (Always-On)：消息入口在 WhatsApp/Telegram 等渠道，意味着它得像服务一样 24/7 在线；笔记本的休眠/移动性反而是负担。

- **安静省电**：长时间跑 Node 服务、偶尔跑容器或浏览器自动化，噪音和电费都很敏感；这就是 Mac mini 这类小主机受欢迎的原因。

- **深度生态集成**：在 macOS 上可以更顺滑地用 `system.run` / `system.notify` 这类节点对接系统层能力（通知、快捷指令、iMessage 等），自然就更“像助手”。

个人服务器的再流行

SaaS 时代大家把“运行”交给云，把“数据”交给厂商。Moltbot 这类东西把问题翻回来：你想要可控、可持续、可审计的执行链路，就得有一台**属于你自己的常驻机器**。它不一定是什么“对话式操作系统”，但确实把“个人服务器”这件事重新拉回大众视野：一台小主机 + 一个常驻网关 + 一套工具权限边界，就能把很多琐碎自动化收拢到同一个入口里。

颠覆 SaaS 订阅

Moltbot 的攻击点不在“替代某个单一产品”，而在于它直面了 SaaS 工具栈的两个老毛病：**订阅碎片化**和**数据/流程割裂**。你用 Zapier^[9] 串自动化、Notion^[10] 管知识、Buffer^[11] 发内容、Calendly^[12] 排日程……每个服务各收一份钱，还各自关着自己的数据和流程。Moltbot 的思路是：把入口统一到一个本地网关里，让自动化以“工具调用”形式组合，而不是靠一堆互不相干的 SaaS 拼图。

它展示的路径大致是三条：

- **功能替代（在能模拟人类操作时）**：有浏览器自动化（Puppeteer/CDP）和本机文件能力，很多任务不一定非要 SaaS 的 API——能“像人一样做”，就能覆盖一部分订阅工具的价值。
- **成本结构变化**：从“按人头/按月”变成“按调用/按量”（主要是模型 token + 任务运行时开销）。低频但复杂、强定制的场景，往往更划算。
- **数据更靠近你**：会话、记忆、执行日志默认落在本地目录（例如 `~/.moltbot`），至少在架构上把“数据主权”往用户侧拉了一步——当然这也意味着你得把本地磁盘当敏感资产来管。

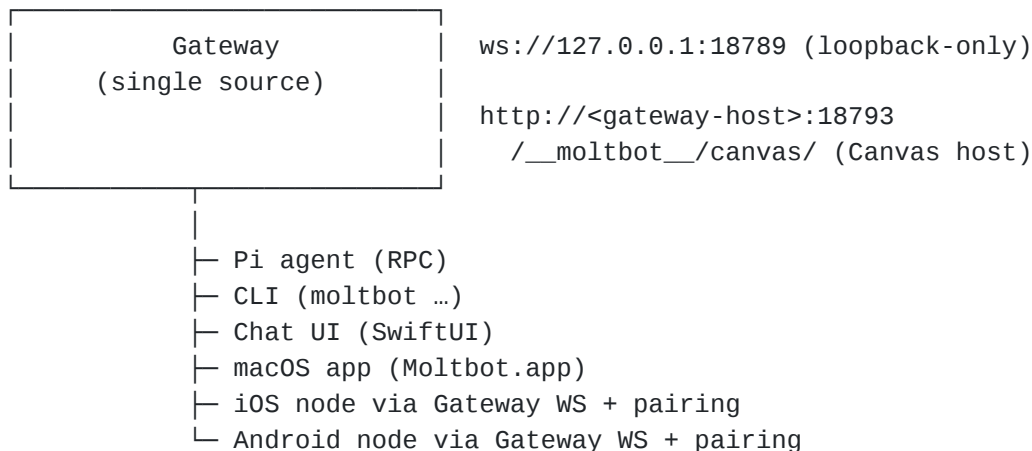
技术架构

Moltbot 之所以能引发技术圈的狂热，不仅因为其概念新颖，更因为其精巧且模块化的技术架构。它不是一个简单的聊天机器人脚本，而是一个复杂的分布式系统网关。

网关架构：代理中枢神经

Moltbot 的核心组件是 Gateway。这是一个基于 Node.js（要求 v22+）的长运行进程，通常监听在 `127.0.0.1:18789`。

Channels: WhatsApp / Telegram / Discord / iMessage / ... (+ plugins)



WebSocket 控制平面

Gateway 维护了一个全双工的 WebSocket 控制平面。这与传统的 HTTP 请求-响应模型有本质区别。

- **实时总线**：所有的客户端（Web UI、Mac 菜单栏应用、移动端节点）、所有的工具（Tools）以及所有的事件（Events）都通过这个 WebSocket 总线进行通信。
- **状态同步**：当用户在手机上发送一条消息，这个状态变更会实时推送到桌面端的 Dashboard 和所有连接的客户端。这种架构保证了多端协同的一致性。
- **事件驱动**：Gateway 不仅仅是被动响应消息，它还能监听系统事件（如文件变动、定时任务触发）。这意味着 Moltbot 可以主动发起对话。例如，当检测到服务器 CPU 负载过高时，Moltbot 可以主动向用户的 Telegram 发送警报。

“Loopback-First” 网络模型

Moltbot 的网关（Gateway）默认仅绑定在 Loopback 接口（127.0.0.1），这一看似简单的设计，实际上体现了对最小暴露面原则（Principle of Least Exposure）的严格执行。

- **安全隔离**：Loopback 绑定意味着，只有运行在本机的进程才能访问 Gateway，局域网其他设备甚至公网请求都会被默认拒绝，从而极大减少了意外暴露或端口扫描攻击的风险。
- **远程访问方案**：为了实现远程控制，Moltbot 官方强烈推荐使用 Tailscale Serve^[13] 或 Cloudflare Tunnel^[14] 等零信任网络技术，而非直接进行端口映射或开放公网端口。这种架构将认证、加密和访问控制委托给更成熟的网络基础设施，从根源上避免将 Gateway 暴露于易受攻击的环境中。

这一网络模型可称为“Loopback-First”——即默认闭合，仅在用户明确配置可信路径时才进行有限开放。这不仅是安全策略，也体现了 Moltbot 本地优先（Local-First）设计哲学的延伸。

🔴 Loopback

Loopback（环回）最简单的理解：**127.0.0.1** 就是“这台机器自己”。访问 **http://127.0.0.1:3000** 等于连回本机进程，不经过外网也不经过局域网。

因此，服务如果只绑定/监听在 **127.0.0.1** (loopback)，就只能被本机访问；同一 Wi-Fi 的其他设备也连不上。反过来，绑定到 **0.0.0.0** 或 **192.168.x.x** 就意味着对外可达，暴露面立刻变大。

像 Moltbot 这类带“控制面 + 工具执行”能力的系统，默认 loopback 是一种 **Loopback-First** 安全默认：先把入口关在本机，避免误上公网；真要远程访问，再显式选择更安全的方式（VPN、SSH Tunnel、Tailscale 等）去“有控制地打开门”。

一句话：连都连不上，比事后补防火墙/鉴权更可靠。

连接层：多渠道聚合

Moltbot 致力于“在你所在的地方”（Where you are）。它通过模块化的 Channels 系统聚合了几乎所有主流通讯协议。

协议适配与模拟

Moltbot 支持的渠道列表令人印象深刻，包括 WhatsApp, Telegram, Slack, Discord, Google Chat, Signal, iMessage 等。为了实现这一点，它采用了多种技术手段：

- **WhatsApp (Baileys)**：WhatsApp 没有开放完整的 Bot API。Moltbot 使用 Baileys^[15] 库来模拟 WhatsApp Web 协议。Gateway 在本地运行一个无头浏览器环境或 WebSocket 模拟层，生成二维码，用户通过手机扫描登录。这使得 Moltbot 能够以“用户本人”的身份收发消息，而不仅仅是作为一个受限的商业机器人。
- **Telegram (grammY)**：利用 Telegram 完善的 Bot API (grammY^[16])，Moltbot 实现了高级特性，如“Draft Streaming”（草稿流）。当 AI 正在思考或生成长回复时，用户可以在 Telegram 的输入框上方看到 AI 实时的打字状态或生成的草稿，极大地优化了交互体验。
- **iMessage (imsg)**：在 macOS 上，Moltbot 调用本地的 imsg^[17] 命令行工具或通过 AppleScript 桥接，实现了对蓝色气泡的控制。这是实现“手机控制家中 Mac”的关键链路。
- **Signal (signal-cli)**：基于 signal-cli^[18] 实现。
- ...

移动端节点与分布式架构

为了应对移动设备无法直接运行完整 Node.js 代理环境的限制，Moltbot 采用了 **Gateway-Node 分布式架构**。移动设备作为具备感知与交互能力的节点，与运行在服务器或本地电脑上的 Gateway 协同工作。

核心架构分离：

- **Gateway（智能核心）**：运行完整的代理环境，负责逻辑推理、上下文记忆、多模型调度和通道管理。
- **移动节点（感知前端）**：iOS/Android 应用通过 Gateway WebSocket 协议连接，作为具备硬件能力的节点而非被动终端。

节点与 Gateway 之间是双向通信关系：

- **Gateway 主动调用节点能力**，通过 node.invoke 调用移动端硬件能力。如：
 - **位置服务**：location.get 获取 GPS 坐标

- 相机功能：camera.snap/camera.clip 拍照或录制视频
- 屏幕录制：screen.record 录制屏幕
- 节点主动向 **Gateway** 发起请求，如：
 - 语音唤醒：通过麦克风检测唤醒词，触发 agent.request
 - 语音转录：发送 voice.transcript 事件
 - 聊天交互：使用共享会话密钥进行消息收发

实际工作流程：

用户："我现在在哪里？"

→ Gateway 接收并处理

→ Agent 决策调用 node.invoke("location.get")

→ 移动节点获取 GPS 坐标

→ Gateway 结合地图 API 返回地理描述

这种架构既保持了移动设备的轻量级特性，又通过分布式协作实现了完整的智能代理能力。

代理能力：从文本到行动

Moltbot 与 ChatGPT 的最大区别在于其执行层。

实时渲染 & A2UI 协议

A2UI^[19] (Agent-to-User Interface) 是 Moltbot 引入的一项革命性技术，位于 vendor/a2ui^[20] 目录中。

- **问题背景**：传统 AI 输出多为文本/Markdown，遇到图表、表单、地图等“可交互复杂界面”时表达力和可用性都不够。
- **生成式 UI**：Moltbot 在这类场景下不直接产出可执行的 HTML/JS，而是产出声明式 JSON UI 描述（组件树/props/数据/事件意图）。宿主端的 A2UI Renderer 读取该 JSON，并通过本地的组件目录（catalog）把它映射成真实 UI；不同平台可用不同 renderer（Web、Flutter、SwiftUI、Jetpack Compose 等）。
- **安全性与一致性**：因为传的是数据 + 受控组件映射，而非任意代码执行，可显著降低 XSS/注入面；同时 renderer 可做 schema 校验、组件白名单、URL/资源策略、权限拦截来兜底。这样同一份 UI 蓝图能跨端一致渲染，并在对话中按需拼装成临时的“微应用（Micro-Apps）”完成任务。

记忆压缩 & 上下文管理

LLM 的上下文窗口是昂贵的资源。Moltbot 采用了一套基于文件的记忆管理机制。

- **会话日志**：所有对话以 JSONL 格式存储在 `~/.moltbot/agents/<agentId>/sessions/`，元数据存储在 sessions.json。
- **自动压缩（Compaction）**：当会话长度超过阈值，Moltbot 会自动压缩旧对话为摘要，保留最近消息。压缩前会触发静默记忆刷新，将关键事实写入 `memory/YYYY-MM-DD.md` 和 `MEMORY.md`。这种机制确保了代理能够长期记忆重要信息，同时保持推理成本可控。
 - **memory/YYYY-MM-DD.md**：每日日志（仅追加），会话开始时阅读今天和昨天的内容。

- **MEMORY.md**：精心设计的长期记忆，仅在主会话、私有会话中加载（绝不在群组上下文中加载）。

生态扩展

Moltbot 的“可扩展性”主要来自三层可插拔面：**Skills** 用来“教会模型如何调用某个工具”，**Plugins** 用来“给网关/CLI/工具面新增能力”，而 **MCP** 这类协议生态，通常是**通过 Skill/Plugin 做桥接**接入 Moltbot。Moltbot 基于 `@agentclientprotocol/sdk` 实现，并非标准的 MCP Client。

注：`@agentclientprotocol/sdk` 不是 MCP 的实现，它实现的是 **ACP (Agent Client Protocol)**；ACP 仅在可能的地方复用 MCP 的 JSON 表示，并扩展了面向“编码代理/编辑器 UX”的类型（如 diff 展示等）。

原生技能 (Native Skills)

把“工具使用方法”装进 SKILL.md。原生技能是 Moltbot 最核心、也最轻量的扩展单元：**一个技能就是一个目录**，至少包含一份带 YAML 头信息的 **SKILL.md**，外加脚本/资源文件，用来告诉模型“什么时候用、怎么用”。

- **结构与加载**：Skills 会从 *bundled* / 本机共享 / 工作区 三个位置加载，并遵循明确的覆盖优先级 (*workspace* > *local* > *bundled*)。
- **热更新体验**：开启 skills watcher 后，**SKILL.md** 变更会触发快照刷新；在“下一次 agent turn”即可拾取更新，接近热加载。
- **生态来源**：官方/社区通常通过公共技能仓库分发与同步（如 MoltHub^[21]、awesome-moltbot-skills^[22]）。
- **安全边界**：技能本质上是“可信代码 + 指令”，第三方技能应当先审计再启用；密钥注入与沙盒策略也需要明确规划。

插件 (Plugins)

把能力注入网关与工具面 (Commands / Tools / Gateway RPC)。当你需要的不只是“教模型怎么用工具”，而是要**新增工具、命令、后台服务或网关 RPC**，就该用插件：插件是一个可分发的代码模块，可通过 CLI 安装/启用，并在网关侧生效。

- **能力范围**：Plugins 可以扩展命令、工具与 Gateway RPC，适合“可维护、可版本化、可选装”的中大型能力。
- **与 Skills 协同**：插件也可以自带 skills 目录；启用插件后，这些技能会像普通技能一样参与加载与优先级规则。

MCP (Model Context Protocol)

用“标准协议生态”接入外部工具，但通常要先做桥接。MCP 的价值在于把“Client ↔ Server (工具/数据源)”的对接标准化：只要实现 MCP Client，就能以统一方式连接任意 MCP Server。

- **Moltbot ≠ 天生的 MCP Client**：Moltbot 有自己的网关协议与扩展体系；要吃到 MCP 生态，通常是用一个 Skill/Plugin 把 MCP Client 能力桥接进来。

- **典型桥接方式**：把“MCP Client/调用器”当作一个工具接入（例如 `mcpporter` 这类 CLI：用于列出/配置/认证/调用 MCP servers/tools，支持 HTTP 或 stdio）。
- **什么时候选 MCP**：当目标是对接重型、持久化、跨产品复用的数据源/系统（如企业 SaaS、数据库、知识库等），MCP 的“标准化接口 + 工具清单/Schema”会更适合长期演进；而快速脚本化、临时自动化仍然更偏 Skills（如网页抓取、本地文件操作、简单计算）。

权限 & 安全

当 Moltbot 获得 Shell/文件/网络/通道能力后，安全就从“防数据泄露”升级为“防主机级失陷”。官方把这类能动真工具的个人代理形容为 **spicy**：一旦被 prompt injection / 社工诱导成功，风险不再是回答错，而是工具被滥用，后果取决于你给了它多大权限与作用域。

所谓**爆炸半径 (blast radius)**：如果攻击者间接“控制”了 Moltbot，它能做到什么。只要允许执行与读写，最坏可以删文件、读密钥、植入持久化；配对了 macOS node 时还可能通过 `system.run` 在那台 Mac 上远程执行命令——这必须按“运维权限”来管理。

常见事故更多是“脚枪配置”而非 0day。比如入站没锁死：未知 DM 应走 **pairing**（陌生人只拿到短码，消息不处理，审批后才放行）；群组应 `requireMention + allowlist`。再比如反向代理/公网暴露：如果把 Gateway 放到 Nginx/Caddy 后，必须正确配置 `gateway.trustedProxies`、确保代理覆盖 `X-Forwarded-For`、并阻断对 Gateway 端口直连，否则可能出现“外部请求被误判为本地信任”的风险。Control UI 也应避免使用 `allowInsecureAuth` 这类降级开关（只用于短期排障）。

另外两条红线：**插件基本等同“同权代码”**（供应链与安装脚本都要按运行不可信代码审计），以及**磁盘就是信任边界**——会话转录会落盘

到 `~/.moltbot/agents/<agentId>/sessions/*.jsonl`，任何能读你磁盘的进程/用户都可能读到日志；因此要把 `~/.moltbot` 权限收紧，并在需要强隔离时用不同 OS 用户或不同主机跑不同 agent。

落地思路是零信任三段式：先做 **Identity**（谁能说话：`pairing/allowlist/open`）、再做 **Scope**（它能在哪些会话/哪些工具里动手：`mention gating`、工具策略、沙箱）、最后才是 **Model**（默认会被诱导，把伤害限制在可控范围）。

Moltbot 提供的防线也对应这三段：DM 配对用 `moltbot pairing list` 查看请求、`moltbot pairing approve <channel> <code>` 放行；安全基线用 `moltbot security audit / --deep` 做检查、`--fix` 自动收紧（通常会把关键目录/文件权限压到 700/600 等）；工具执行隔离用 **Docker sandbox** 控爆炸半径（`mode: off/non-main/all`；`scope: session/agent/shared`；`workspaceAccess: none/ro/rw`；默认网络可设为 `none`；注意标记为 `elevated` 的工具可能绕过沙箱，需要严格控白名单）。

安装指南

注：目前 Moltbot 项目仍在高速迭代，文章内容随时可能过时（尤其是更名风波导致文档配置仍在更新中）。建议参考官方文档获取最新安装与配置指南：Moltbot Docs^[23]。

推荐使用官方一键安装脚本。macOS/Linux 直接跑安装器即可，它会引导你完成安装与后续向导；Windows 则提供 PowerShell 安装脚本（但实际运行强烈建议在 WSL2 内按 Linux 流程走）。

```
# macOS / Linux
curl -fsSL https://molt.bot/install.sh | bash # Windows PowerShell
iwr -useb https://molt.bot/install.ps1 | iex
```

安装器支持“自动化/CI 友好”的参数与等价环境变量。常用的就是：选择 npm 或 git 安装、指定 git 目录、跳过向导、禁用交互、dry-run 预演；环境变量可用于无交互部署。例如：`--install-method npm|git`、`--git-dir`、`--no-onboard`、`--no-prompt`、`--dry-run`（以及 `CLAWDBOT_*` 环境变量）。了解更多 cli-flags^[24]。

全局包管理器安装（npm/pnpm）是“可选路径”，但要注意风险。Getting Started 里给了 `npm install -g moltbot@latest` / `pnpm add -g moltbot@latest`。但 2026-01-27 有报告称 npm 上的 moltbot 包名被抢注/冒用（#2775^[25]，邮箱是 @163.com，大概率是被国人抢注了），因此在官方确认修复前，更稳妥的做法是优先用安装器的 git 安装模式（绕开 npm），或明确使用官方仓库源码安装。

开发者/贡献者：从源码安装。这条路径适合需要改代码、调试 UI 构建或自定义运行时的人，流程就是 `clone` → `pnpm install` → `build` → `运行向导并安装守护进程`。

服务器/NAS：Docker 部署。官方给了“快速启动脚本”`./docker-setup.sh`，会构建镜像、跑向导、启动 compose、生成并写入 gateway token 到 `.env`；也提供手动 compose 流程。

更偏运维自动化：Ansible^[26] 一键部署。官方提供独立仓库的一键安装脚本，主打“防火墙优先 + Tailscale 安全远程访问 + Docker 隔离”。

结语

Moltbot 很强，也很危险！它可能是最接近现实的 AI 助理。最直接的感受可能是：成本正从“订阅”迁移到“按量 token”，形态也从“云端聊天”走向“本地可执行”。

但智能的前提是中枢模型得够稳：一次决策跑偏，省下的时间很容易被返工和事故吃回去。**配对了，它就是能长期跑的助手；配错了，就等于把一台带网的终端开给陌生人。**

Skills、ACP、A2UI 等看起来是“新名词”，但在 Moltbot 里却不是摆设：它们被用来**封装工具、呈现交互、对接能力**，组成一个相对完整的可落地代理框架，值得深入学习研究。

我也愈发有种感觉：LLM 越强，越依赖所能调用的底层 API 能力，也更坚定了我做 Noi 的决心（[Noi v1.1.0 发布，Agent 能力初显](#)）。

这篇文章怎么写都不太满意，就先这样吧。还是建议大家多看文档，它的玩法全在配置里...

再拉个交流群吧，公众号发送“molt”获取二维码，感觉 Moltbot 可以玩出很多有趣的东西。

References

[1]

ACP:<https://github.com/agentclientprotocol>

[2]

A2UI:<https://github.com/google/A2UI>

[3]

#1795:<https://github.com/moltbot/moltbot/pull/1795>

[4]

Shodan:<https://www.shodan.io>

[5]

Censys:<https://search.censys.io>

[6]

Moltbot:<https://github.com/moltbot/moltbot>

[7]

Peter Steinberger:<https://steipete.me>

[8]

Moltbot Lore:<https://docs.molt.bot/start/lore>

[9]

Zapier:<https://zapier.com>

[10]

Notion:<https://www.notion.com>

[11]

Buffer:<https://buffer.com>

[12]

Calendly:<https://calendly.com>

[13]

Tailscale Serve:<https://tailscale.com>

[14]

Cloudflare Tunnel:<https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel>

[15]

Baileys:<https://github.com/WhiskeySockets/Baileys>

[16]

grammY:<https://github.com/grammyjs/grammY>

[17]

imsg:<https://github.com/steipete/imsg>

[18]

signal-cli:<https://github.com/AsamK/signal-cli>

[19]

A2UI:<https://github.com/google/A2UI>

[20]

vendor/a2ui:<https://github.com/moltbot/moltbot/tree/main/vendor/a2ui>

[21]

MoltHub:<https://clawdhub.com>

[22]

awesome-moltbot-skills:<https://github.com/VoltAgent/awesome-moltbot-skills>

[23]

Moltbot Docs:<https://docs.molt.bot>

[24]

cli-flags:<https://docs.molt.bot/install/index-flags>

[25]

#2775:<https://github.com/moltbot/moltbot/issues/2775>

[26]

Ansible:<https://docs.molt.bot/install/ansible>